Project: Productionizing your LLM Server

Due: 02/14/2026 11:59 PM
Extended Deadline: 02/15/2026 11:59 PM

Introduction
- Objectives
- Context & Prerequisites
- Evaluation Method
- Handout

Refactor to FastAPI
- Task Overview
- Task

Testing and pre-commit hooks
- Task Overview
- Task

CI pipeline for FastAPI
- Task Overview
- Task

Wrap Up
- Conclusion

CI pipeline for FastAPI (Task)

Continuous Integration to Completion

Building upon the previous task where you developed well-tested code using mocking strategies, this task focuses on establishing a comprehensive CI pipeline. The objective is to create an automated continuous integration workflow that ensures code quality, security, and deployability.

Pipeline Requirements

The CI pipeline will execute the following stages in sequence:

1. Validate code linting and formatting standards
2. Execute mypy for type annotation verification
3. Run pydoclint to ensure docstring correctness
4. Execute the complete test suite
5. Generate and report code coverage metrics
6. Build the distribution wheel file
7. Construct the Docker image
8. Perform vulnerability scanning on the image
9. Push the Docker image to Azure Container Registry (ACR)

While you're already familiar with tasks 1-7 and 9, this assignment focuses on integrating all these components, including the new vulnerability scanning step, into a cohesive CI pipeline.

For this project, we'll utilize GitHub Actions as our CI/CD platform. In Cloud DevOps, both Azure DevOps and GitHub Actions proved effective due to their:

> Straightforward configuration and extensive library of pre-built actions
> Intuitive graphical user interfaces for monitoring and management

GitHub Actions will serve as our platform for this implementation.

Pre-requisite: Local Development Setup

In the previous task, pre-commit hooks enforced quality standards before each commit. For this project, we're transitioning these checks to the CI pipeline to provide greater flexibility in your local development workflow. This approach allows you to commit frequently and push batches of changes when ready, without the overhead of running hooks on every commit. While pre-commit hooks are valuable, they can sometimes impede rapid development iteration.

Note: Removing pre-commit hooks is a recommendation, not a requirement. If you prefer to retain them in your workflow, you may do so without penalty.

Execute the following commands to uninstall the pre-commit hooks:

```
poetry run pre-commit uninstall
rm ./.pre-commit-config.yaml
```

Irrespective of whether you removed pre-commit hooks or not, add the necessary linting tools to your development dependencies:

```
poetry add --group dev ruff pydoclint
```

Before proceeding, verify that your codebase meets all quality standards by running:

```
poetry run ruff check --fix --output-format=c
poetry run ruff format
poetry run pydoclint --style=google --exclude
```

Please address any violations reported by these tools before continuing with the CI pipeline implementation.

Commit any new changes you have made.

Subtask 1: Hello Terraform

In the previous project, you utilized Terraform to provision and manage cloud infrastructure, specifically creating:

- A resource group
- An Azure Container Registry (ACR)
- A Kubernetes cluster
- Role assignments enabling the Kubernetes cluster to access the ACR

Let's begin by recreating this infrastructure.

Execute the following commands to initialize and apply your Terraform configuration:

```
cd terraform
terraform init
terraform apply
```

Important: Carefully review the execution plan that Terraform generates before confirming the apply operation. This ensures the provisioned resources match your expectations.

Troubleshooting

If you encounter issues during provisioning:

1. Navigate to the Azure portal and manually delete the resource group
2. Retry the `terraform apply` command

If problems persist:

1. Remove all Terraform state and cache files:
   - `.terraform/` directory
   - `.terraform.lock.hcl` file
   - `.tfstate` files
2. Delete the resource group in Azure portal
3. Rerun the initialization and apply commands

If issues continue after these steps, attempt to debug the specific error messages. Don't hesitate to reach out on Ed Discussion for assistance.

Integrating GitHub Actions with ACR

With the infrastructure deployed, the ACR can now serve as the storage location for Docker images produced by your GitHub Actions CI pipeline.

For GitHub Actions to push images to the ACR, the repository requires access credentials (username and password). However, since you'll be reusing this Terraform configuration and code repository throughout the course, you'll need to destroy and recreate the ACR multiple times. Each recreation generates a new auto-generated password, and manually updating GitHub secrets after each deployment would be inefficient.

We'll leverage the Terraform GitHub provider to automate this process. This approach will:

- Generate ACR credentials automatically
- Create GitHub Actions secrets programmatically
- Eliminate manual secret management

Add your GitHub organization and repository information to the Terraform variables:

```
variable "gh_org" {
  description = "Org for the handout repo"
  type        = string
  default     = "SAIL-Classroom" # The org in
}

variable "gh_repo" {
  description = "Name of the github repo"
  type        = string
  default     = "cloud-native-handout-<Github
}
```

Include the GitHub provider in the `required_providers` block of your `main.tf`:

```
required_providers {
  # ... keep other providers as is
  github = {
    source  = "integrations/github"
    version = "6.9.0"
  }
}
```

Add the provider configuration block in your `main.tf` alongside other provider declarations:

```
provider "github" {
  owner = var.gh_org
}
```

Run the initialization command to download and prepare the GitHub provider:

```
terraform init
```

Terraform requires authentication to interact with GitHub. Multiple authentication methods are available - refer to the [official documentation]() for details.

We'll use the GitHub CLI (`gh`) authentication method, which provides a streamlined authentication experience.

Execute the following commands to install the GitHub CLI:

```
curl -fsSL https://cli.github.com/packages/gi

echo "deb [arch=$(dpkg --print-architecture)

sudo apt update

sudo apt install gh -y

gh --version
```

Alternative installation methods are available in the [official installation documentation](#).

Once installed, authenticate using the following command and select the web browser login option:

```
gh auth login
```

This method eliminates the need for Personal Access Tokens, as Terraform will automatically utilize the GitHub CLI's authentication credentials.

> **Warning**
>
> You must install gh cli and the submitter uses it to authenticate and verify the status of github actions. If you are having issues with the browser-based authentication, you may use a Personal Access Token (PAT) but ensure that it has access to list your repo's and github action jobs.

Let's add the logic to generate a password for the ACR and pass it to GitHub Actions.

ACR provides a variety of methods for authentication. The most [production-grade method](#) is to use a service principal. A service principal is an automation user that has an ID and secret key. You can assign a role to the service principal to enable read access to your ACR. However, creating a service principal requires permissions to manage Azure AD, which students in this course don't have. Therefore, we'll use a simpler method: admin username and password.

Set `admin_enabled` to `true` in your "acr" resource. It should look like this:

```
resource "azurerm_container_registry" "acr" {
  name                = var.acr_name
  resource_group_name = azurerm_resource_grou
  location            = azurerm_resource_grou
```

```
  sku                 = "Standard"
  admin_enabled       = true
}
```

Your task is to add a `github_actions_secret` resource that takes the name of your repository from the variables in `variables.tf`, sets the secret name to `ACR_TOKEN`, and specifies a `plaintext_value` equal to the `admin_password` of the ACR. Since the `admin_password` returns a sensitive string, you'll need to wrap it with the `nonsensitive` function before passing it to `plaintext_value`.

Refer to the GitHub provider documentation for guidance.

Apply your changes and navigate to your GitHub repository > Settings > Secrets and variables > Actions to verify that a repository secret has been created.

Warning

Note that by converting the sensitive admin password to nonsensitive plaintext, the Terraform state file now contains the password in plaintext. This is why Terraform state files must always be considered sensitive and stored with extreme caution.

Subtask 2: Github actions in action

Now we should be able to use the ACR password via the repository secrets in our GitHub Actions. Let's start writing our CI pipeline.

Create a `main.yml` file in `.github/workflows` in the project root.

The following is some starter code for your convenience:

```
name: Build and Push to ACR
on:
  push:
    branches:
      - master
jobs:
  buildAndPush:
    runs-on: ubuntu-latest
    env:
    defaults:
      run:
        working-directory: llm_server
    steps:
```

```
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install pipx
        run: |
          python -m pip install --user pipx
          echo "$HOME/.local/bin" >> $GITHUB_

      - name: Install Poetry with pipx
        run: |
          pipx install poetry==2.2.1
```

In the above code, we trigger the pipeline on every push to the master branch. We set the runner to ubuntu-latest and add a working-directory so that we don't have to `cd` into `llm_server` in every step. The steps checkout the repository, set up Python, install pipx (a utility that helps install Python-based CLIs globally and is the recommended method from the Poetry documentation for installing Poetry in CI environments), and use pipx to install Poetry with a specific version number for consistency.

Your first task is to add two environment variables:

1. `ACR_LOGIN_SERVER` which is the `<your_acr_name>.azurecr.io` URL
2. `ACR_USERNAME` which is the part before `azurecr.io`

Your task now is to add the following stages to the CI pipeline:

1. Poetry install
2. Run mypy
3. Run ruff check, ruff format, and pydoclint (make sure you're only checking and not fixing - remove `--fix`)
4. Run the tests and print the coverage report

Take this time to commit your changes and test your newly created CI pipeline. Make sure that all the stages pass.

Warning

Please add the commands you want to run directly to the .yml file, do NOT use a shell script that runs the commands for you. The submitter uses the commands from the .yml file to grade this task.

Subtask 3: Build and Push

Moving forward, your next task is to build and push the Docker images to the ACR.

Add the following steps:

1. Use Poetry to build the wheel file - `poetry build --clean --format wheel`
2. Download the model file - `wget https://huggingface.co/TheBloke/TinyLlama-1.1B-Chat-v1.0-GGUF/resolve/main/tinyllama-1.1b-chat-v1.0.Q2_K.gguf`
3. Build the Docker image. Make sure you correctly tag the Docker image. It should adhere to the format `$ACR_LOGIN_SERVER/llmapp:latest`
4. Login to ACR using `docker login`, specifying the login server `$ACR_LOGIN_SERVER`, username `$ACR_USERNAME`, and password `${{ secrets.ACR_TOKEN }}`

If you observed, we are using the secrets that we added using Terraform here to login.

Finally, add the last step:

> Push to ACR

Subtask 4: Scanning the image

Now that we have a system that can automatically build and push our image, let's add a task to scan our Docker image for security issues, which we can then decide to update at our convenience.

To do so, we will use Grype, a tool purpose-built for this purpose. It will give us a list of known vulnerabilities for us to look at, which will allow us to see what our container's current security looks like.

Here we are using an existing GitHub Action that provides this functionality out of the box.

Scan action

Your task is to read this documentation and add a step to the CI pipeline just after you build the Docker image. This stage should take the same image name that you're using to build the image. It should set `fail-build` to `true` so that the pipeline fails if a critical security vulnerability is detected in our container. This can be tuned by setting the `severity-cutoff` to `critical`. Note that you can't use `$ACR_LOGIN_SERVER` to refer to your environment variables here, as this is not inside the `run` section but is a parameter for the YAML itself. Therefore, consider using `${{ env.ACR_LOGIN_SERVER }}`.

Make sure to commit and check if the pipeline passes all the stages successfully.

Submission

```
./submitter -t task3
```

Make sure to also commit the code to your GitHub.

Reflection Questions

In `answers/unit1answers.txt`, briefly answer the following questions. Answer in your own words without looking things up online or using GenAI tools.

1. Your source code worked great in the preparing for Cloud Native project. Explain 3 possible reasons we decided to "fix that which isn't broken".
2. We decided to switch from Flask to FastAPI. Explain 2 benefits FastAPI gives us over Flask.
3. You've verified from Project 0 that your application works just right. Why do we still need automated testing?
4. Assuming your organization forbids you from using GitHub-hosted runners to run your CI pipeline, what alternate approach can you use?

← Previous Page Next Page →

Report an Issue? © 2026 Copyright Sail() Platform